Bruce

Household AI — Master Specification

bruce.app

Conception — April 2026

Prepared by Jake Johnson
Foundation Insurance Group | Capital Petsitters

01 Overview

Bruce is a private household AI built for the Johnson family. It is powered by the Anthropic Claude API and lives at bruce.app. Bruce is not a product — it is infrastructure built for one household, designed to feel like a trusted family presence rather than a generic AI tool.

The closest reference point is Claude.ai — same bones, same clean interface, same project and chat structure. Bruce extends that foundation with one layer Claude does not have: a shared household dimension with controlled access, group conversations, family collaboration, and household-specific memory.

Bruce has a core identity that stays consistent across every interaction: calm, reliable, consistent, intelligent, and caring. His tone shifts naturally based on who he is talking to and what context he is in — but he is always recognizably Bruce.

02 Household

Active Members

Jake Johnson	Admin — 36. Manages all AI infrastructure, settings, and build. Account executive at FIG and co-owner of Capital Petsitters.
Laurianne	Member — 33. Full private workspace. Shared access to family projects and group chat.
Jocelynn	Member — 16, treated as adult. Full private workspace. Shared access to family projects and group chat.
Nana	Member — 69. Full private workspace. Lives nearby, not in household full time. Shared access to family projects and group chat. Co-owner of Capital Petsitters.

Household Context (non-account)

Elliot (8), Henry (5), and Violette (5) exist in the household shared memory as context — names, ages, relevant family facts — but do not have accounts or interact with Bruce directly.

Authentication

- Google OAuth — single sign-in button, no passwords
- Face ID handled natively by Google on iPhone
- Invitation only — new members added by Jake from admin panel
- Invite link is single-use, expires in 48 hours

- No setup flow on first login — Bruce opens ready to use

Privacy Principle

Privacy is enforced at the database level using Supabase Row Level Security, not at the UI level. Every piece of content has an owner ID. The database checks authenticated identity before returning any data. No member — including the admin — can access content from another member's private space. This is an architectural guarantee, not a setting.

03 Interface

Design Language

- Claude cousin — same clean minimal structure, same spatial logic
- Light and dark mode — follows device system setting automatically
- No logo, no wordmark, no branded orb — Bruce is the name, not a brand
- Accent color: deep blue-green (#0F6E56) — appears in active states, send button, highlights, selected items
- Mobile primary design, desktop fully supported, same experience on both

Access

- Progressive Web App — installs to iPhone home screen, feels native, no App Store required
- Available from anywhere in the world on any connection
- Cloud hosted on Vercel at bruce.app — always on, always reachable
- Offline behavior: read-only — browse cached conversations and project files, no new messages until connection restored

Navigation — Desktop

- Persistent left sidebar — always visible, never changes based on current screen
- Sidebar sections: Projects (with member pips), Chats (standalone only, flat list), Family (pinned at bottom)
- Projects and chats are fully separated — project chats never appear in the standalone chat list
- User profile and settings at the bottom of the sidebar

Navigation — Mobile

- Bottom nav handles primary navigation: Home, Chats, Projects, Family
- Sidebar becomes a slide-out drawer triggered by hamburger menu top left
- Tap dimmed area to dismiss drawer
- Inside a chat or project: bottom nav disappears, top bar with back arrow handles navigation

Welcome Screen

When a member opens Bruce with no active chat, they see a personalized greeting and four contextual suggestion cards surfaced from their recent activity and upcoming context. No tutorials, no feature tours — Bruce is ready.

04 Chats

Standalone Chats

- Private one-on-one conversation between the member and Bruce
- Flat list in sidebar, most recent at top, no date groupings
- No project context loaded — general purpose
- Incognito mode available via toggle in the chat topbar (see Section 4.3)

Adding People to a Standalone Chat

- Any member can add other household members to a standalone chat from the topbar
- Tap Add People — a member picker drops down showing available household members
- Selected members receive a push notification they have been added
- Chat becomes a named group thread — member avatars appear in topbar
- Input placeholder updates to reflect who is in the chat
- Adding people is not available when incognito mode is on

Incognito Mode

- Toggle in the standalone chat topbar — labeled Incognito with eye icon and toggle switch
- When active: interface desaturates, topbar changes, one-line notice appears below topbar
- Bruce does not save the conversation, update memory, or show it in chat history
- Session ends and disappears completely when the chat is closed
- Available for standalone chats only — project chats cannot be incognito
- Not available when other members have been added to the chat

Group Chats

- Ad hoc group threads between any subset of household members plus Bruce
- Not tied to a project — lightweight and conversational
- Distinct from the permanent family chat
- Created by adding people to a standalone chat

Family Group Chat

- Permanent shared channel — all four household members plus Bruce
- Pinned at the bottom of the sidebar, always in the same place
- Member messages are attributed by name with color-coded avatars
- Bruce is always listening — reads every message, builds context as the conversation flows
- @ mention to bring Bruce in cold, natural triggers once he is active
- Long press any message to surface Ask Bruce as a quick action — hidden until needed
- Pause Bruce toggle in the topbar — play/pause icon with Bruce label
- When paused: Bruce reads but does not respond unless manually triggered
- Brief working indicators: typing dots for short responses, Bruce is working on it for longer tasks, connector-specific label when hitting an integration

Bruce's Group Chat Behavior

Bruce participates in group chats as a presence, not a bot. He uses judgment about when to respond versus stay quiet based on three rules:

- Low stakes actions — add to a list, log a preference, note something simple — Bruce acts and confirms briefly. Example: Added deodorant to the grocery list.
- Medium stakes actions — update a document, modify a project, schedule something — Bruce flags before acting. Example: I can add this to the meal plan. Want me to go ahead?
- High stakes actions — connector write operations, deletions, irreversible changes — Bruce always asks explicitly before acting.

Cross-Context Routing

Bruce can route information from the family group chat into the appropriate project without the message sender gaining access to that project. If Jocelynn mentions she needs deodorant in the family chat, Bruce recognizes it as grocery-relevant, adds it to the meals and groceries project, and confirms in the family chat. Jocelynn never gains access to the meals project.

05 Projects

Structure

- Mirrors Claude Projects — familiar to anyone who has used Claude
- Any member can create a project and invite whoever they want
- No forced admin inclusion — Jake may not be in every project
- Project chats are contained inside the project and never appear in the standalone chat list

Project Home Screen

Opening a project shows a home screen before any chat. The home screen displays:

- Project name, icon, and member avatars
- Instructions — plain English text field, editable, shapes how Bruce behaves in this project
- Files — Google Drive files attached to the project with type icons and last updated dates
- Connectors — active integrations with live/offline status indicators
- Members — who is in the project and their role, with add member option
- Chats — full list of project chats ordered by recency, with last message preview
- New chat button prominent at top right and as first item in the chat list

Project Chats

Selecting a chat from the project home opens a standard chat interface identical to standalone chats. Bruce has full project context loaded — all files, instructions, and chat history within the project. The project name appears in the breadcrumb at the top.

Project Instructions

A plain English text field on the project home screen. Written in natural language — no technical prompting required. Bruce reads it and adjusts his behavior for that project. Instructions are living documents — Bruce updates them automatically as important things emerge in project conversations.

Example for CPS: Business workspace for Capital Petsitters. Be concise and practical. Nana prefers clear simple explanations. Always recalculate WC at 3% from scratch before any payment action.

Sharing from Projects

Two sharing models are available, both initiated by the project owner:

- Snapshot share — read-only view of a specific conversation. Recipient sees the conversation but has no access to the broader project. History before the share is not included.

- Spin-off shared chat — pull a topic out of a project into a new collaborative thread that includes whomever the owner invites. Clean break from the private workspace.

Household Projects (Reference)

CPS — Operations	Jake + Nana. Capital Petsitters business workspace. QuickBooks, Precise Petcare, and Melio connected.
College Prep	Jake + Jocelynn. Ongoing college planning workspace.
Meals & Groceries	Jake + Laurianne. Weekly meal planning, recipe suggestions, and grocery list management.
Family Calendar AI	Shared. Google Calendar integration, household coordination.
Home Renovation	Jake + Laurianne. Home projects and planning.

06 Google Integration

Authentication

- Google OAuth is the only authentication method — no separate passwords
- Each household member signs in with their own personal Google account

OAuth Scopes

Bruce requests the minimum necessary permissions. Scopes are narrow and transparent to the user at sign-in:

- Google Drive — access only to files and folders within the Bruce folder Bruce created, never the whole Drive
- Google Docs, Sheets, Slides — read and write within scoped Drive access only
- Google Calendar — read and write access for event creation and modification
- Google profile — name and email for authentication only

Drive Folder Structure

On first login, Bruce automatically creates the following folder structure at the root of each member's Google Drive:

- Bruce/ — root folder created by Bruce
- Bruce/Personal/ — private files and documents for that member
- Bruce/Projects/ — project-specific files organized by project name
- Bruce/Shared/ — link to the shared family Drive folder

The shared family Drive folder lives in Jake's Drive and is shared natively via Google with all household members. Bruce never touches anything outside his designated folder structure.

Calendar

- Bruce can read all calendar events to surface context in conversations and the morning summary
- Bruce can create, update, and delete events when asked
- Bruce always confirms before creating or modifying a calendar event — medium stakes rule applies

07 Memory

Memory Layers

Household shared	Family-level facts Bruce always knows. Names, relationships, ages, important household context. Bruce manages this autonomously using the same judgment as personal memory. No admin approval gate — only a cleanup tool in the admin panel if something needs correcting.
Personal — Core	Permanent until manually changed. Maximum 20 records per member. Always loaded on every API call. Professional context, strong preferences, important life facts.
Personal — Active	Relevant to the last 30-60 days. Maximum 50 records, top 15 loaded per conversation based on relevance score. Things currently in motion.
Personal — Archive	Older context compressed into summaries. Never auto-loaded — only pulled if Bruce specifically needs historical context. Unlimited records.
Project memory	All chats, files, and instructions within a project. Bruce knows everything inside a project and gets smarter about it over time.

How Memory is Created

- Bruce generates memories automatically at the end of conversations — no manual input required
- He identifies facts, preferences, ongoing situations, and important context and writes them as clean concise statements
- This happens silently in the background — members see memories in their memory view, not in the conversation
- Members can suggest additions to household memory via an in-chat prompt — Bruce surfaces the option when something seems household-relevant

Memory Hygiene

- Relevance scoring: every time a memory is referenced its score increases. Scores decay weekly via a background job. Frequently relevant memories stay active, rarely surfaced ones fade to archive.
- Automatic compression: conversations summarize as they age. Detail fades, insight stays.
- Hard size ceiling enforced per member — system stays lean automatically.
- Scheduled background hygiene: weekly job compresses, deduplicates, and maintains size limits.

Memory Boundaries

- Private memory never bleeds into shared spaces unless explicitly shared in a shared context
- Incognito conversations leave no memory trace
- Each member can view, edit, and delete their own memories from their profile settings

API Memory Budget

The total memory block sent with each API call stays under 500 words. Household shared memory plus personal core memories plus the top 15 active memories by relevance score. This keeps cost and response quality balanced.

08 Notifications

Infrastructure

- Firebase Cloud Messaging — free, works on iOS and Android, handles service worker complexity
- Supabase webhooks trigger reactive notifications on database events
- Cron job handles scheduled proactive notifications

Reactive Notifications

Event-driven — triggered immediately when something happens:

- @ mention in a group chat
- Project invite
- Someone shares a chat or snapshot with you
- Bruce completes a long-running task you requested
- Connector event — QuickBooks sync, Precise Petcare update, Melio payment

Proactive Notifications

Bruce-initiated — two types:

- Morning summary: cron fires at each member's preferred time (default 8am). Generates a personalized summary via API with that member's context loaded. Delivered as a push notification that opens directly into a live Bruce conversation — immediately actionable, not a dead-end digest.
- Proactive nudges: Bruce surfaces things that clear a meaningful threshold. Approaching deadlines, stalled projects, calendar conflicts, unfinished tasks. Not every observation — only what genuinely warrants attention. Reflects his core identity: calm and measured.

Member Controls

- Each member sets their own notification preferences independently
- Household-wide proactive sensitivity setting in admin panel: low, medium, high
- Members override household sensitivity from their own profile settings
- Default morning summary time: 8am, adjustable per member

09 Connectors

Architecture

Connectors are project-scoped — a connector available in CPS is not available in college prep. The connector architecture is extensible, new integrations can be added per project as needed without rebuilding anything.

Active Connectors

Anthropic API	Powers Bruce. Model: claude-sonnet-4-6. All projects.
Google OAuth	Authentication. All members.
Google Drive	File system for all projects. All members.
Google Docs / Sheets / Slides	Read and write. All projects.
Google Calendar	Read and write. All members.
Image generation	Replicate or equivalent. All projects.
Web search	Perplexity or equivalent. All projects.
QuickBooks	CPS project only. Read and write. Payroll and financial data.
Precise Petcare	CPS project only. Read only. Schedule and service data.

Melio	CPS project only. Read and write. Contractor direct deposit and batch payments.

10 Bruce's Identity

Core Character

Bruce has a consistent underlying identity that stays the same across every interaction, every member, and every context. It is expressed differently depending on who he is with and what they are doing — but he is always recognizably Bruce.

Calm	Never reactive, never overwhelming. Steady presence whether the task is a grocery list or a hard family decision.
Reliable	Does what he says. Follows through. Admits when he does not know something rather than guessing confidently.
Consistent	Same Bruce every time, every person, every context — just expressed differently. No drift over time.
Intelligent	Engages seriously. Does not dumb things down unnecessarily. Connects dots across conversations and context.
Caring	Genuinely oriented toward the wellbeing of the family, not just task completion. Notices when something matters beyond the surface request.

Contextual Tone

Jake — dev / build context	Technical, direct, no hand-holding. Peer-level conversation.
Jake — business context	Professional, efficient, business-aware.
Laurianne	Conversational, warm, collaborative.
Jocelynn	Encouraging, peer-like, not condescending.
Nana	Clear, patient, straightforward. No unnecessary technical language.
Family group chat	Relaxed, personable, reads the energy of the conversation.

Project-Level Persona

Each project has a plain English instructions field that shapes how Bruce behaves specifically in that project. Written in natural language, no technical prompting. Bruce reads it and adjusts. Instructions do not affect how Bruce behaves anywhere else.

11 Admin Panel

A separate layer from Jake's personal workspace — not a project, not a chat. Accessible only to Jake. Organized into five sections:

Usage and Cost

- Total API spend this month in dollars
- Total messages, images generated, and documents created
- Per-member message volume shown as a bar chart
- Cost breakdown: API chat, API images, hosting and database
- Content is never visible — only volume and cost

Members

- Full member table: name, email, role, last active, message count
- Edit any member's role or access
- Generate and send new invite links
- Deactivate a member — soft delete with 30-day holding period before permanent purge

System Health

- Status of all services: Anthropic API, Vercel hosting, Supabase database, each connector
- Live or offline indicator with last sync time for each connector
- Model version, average response time, error count in last 24 hours
- Database size and backup status

Memory

- Per-member memory size and tier breakdown (Core / Active / Archive) shown as bar visualization
- Readable tags showing what Bruce remembers at a high level — not full private content
- Manage button to view, edit, or delete specific memory entries per member
- Trigger manual compression for any member if size is growing unexpectedly

Bruce Dev

- Full technical self-context loaded — stack, architecture, schema, active connectors, environment
- Jake's workspace for troubleshooting, debugging, and building
- Paste logs, error messages, and configs — Bruce has full context to respond meaningfully
- Living documentation of the system that gets updated as the build evolves
- This is the workspace where Bruce helps build and maintain himself

12 Technical Architecture

Stack

Frontend	Next.js — React framework, PWA support, server-side rendering
Hosting	Vercel — auto-deploy from GitHub, edge functions, zero-config SSL
Database	Supabase — Postgres with Row Level Security, real-time subscriptions, built-in auth
Authentication	Supabase Auth + Google OAuth — no passwords, no custom auth logic
Background services	DigitalOcean — cron jobs, memory hygiene, webhook handlers, persistent processes
Push notifications	Firebase Cloud Messaging — iOS and Android, service worker, free tier
AI model	Anthropic API — claude-sonnet-4-6
Image generation	Replicate or equivalent
Web search	Perplexity API or equivalent
Domain	bruce.app — primary user-facing domain
Infrastructure domain	creativeoutletcoding.com
Version control	GitHub — push to main triggers Vercel auto-deploy

Data Model (Core Tables)

users	One row per member, linked to Google account via Supabase Auth
projects	Owner ID, name, instructions, privacy level, created date
project_members	Project ID, user ID, role — the join table controlling who is in what
chats	Owner ID, project ID (nullable), type (private / group / incognito), created date
chat_members	Chat ID, user ID — for group chats with multiple members

messages	Chat ID, sender ID, content, timestamp, role (user / assistant)
files	Project ID, Google Drive file ID, name, type, last updated
memory	User ID, content, tier, relevance score, category, created, last accessed
household	Shared household context — one record for the whole family
notifications	User ID, type, content, read status, created
pending_memory	Suggested household memory additions — status pending or resolved

Privacy Enforcement

Row Level Security is enabled on all tables in Supabase. Every query is filtered by the authenticated user's ID before any data is returned. This is enforced at the database level — not the application level. No bug in the front end can expose another member's data. The privacy wall is an architectural guarantee.

13 Phased Build Plan

Bruce is designed to be built in independent phases. Each phase has a clean stopping point and a deployable result. Phases can be paused and resumed without breaking continuity. The Bruce Dev workspace in the admin panel maintains living documentation of what has been built, what decisions were made, and what comes next.

Phase 1 Foundation Weeks 1-2	
1	Supabase project setup — database, auth, Row Level Security policies
2	Google OAuth integration — sign in with Google, user creation on first login
3	Core database schema — users, chats, messages, memory, household tables
4	Vercel deployment pipeline — GitHub integration, auto-deploy from main
5	DigitalOcean server setup for background services
6	bruce.app domain configuration and SSL

Phase 2 Core Chat Weeks 3-4	
1	Private standalone chat interface — Jake only at this stage
2	Anthropic API integration — streaming responses, conversation history
3	Chat history — stored in Supabase, displayed in sidebar
4	Basic personal memory — Bruce generates and loads core memories
5	Welcome screen with personalized greeting and suggestion cards

6	Incognito mode toggle

Phase 3 Projects	Weeks 5-6
1	Project home screen — instructions, files, members, chats, connectors
2	Project chat — context loaded from project instructions and files
3	Google Drive integration — OAuth scopes, auto folder creation, file attach
4	Google Docs, Sheets, Slides — read and write from project context
5	CPS project goes live — Jake and Nana
6	Project-level memory and living instructions

Phase 4 Household	Weeks 7-8
1	Remaining member accounts — Laurianne, Jocelynn, Nana via invite flow
2	Family group chat — member attribution, Bruce as participant
3	Bruce group chat behavior — @ mention, judgment triggers, pause toggle
4	Add people to standalone chats — member picker, group thread
5	Cross-context routing — Bruce routes family chat content to projects
6	Firebase Cloud Messaging — push notifications, morning summary cron

Phase 5 Connectors and Admin	Weeks 9-10
1	QuickBooks connector — CPS project, read and write
2	Precise Petcare connector — CPS project, read only
3	Melio connector — CPS project, batch payment preparation
4	Google Calendar — read and write, event creation and modification
5	Admin panel — usage, members, system health, memory, Bruce Dev
6	Memory hygiene — relevance scoring, decay job, compression

Phase 6 Polish	Ongoing
1	PWA optimization — offline caching, install prompt, service worker
2	Snapshot share and spin-off chat from projects
3	Household memory suggestions — in-chat prompt and admin review
4	Image generation integration via Replicate
5	Web search integration via Perplexity
6	Proactive nudge tuning — sensitivity settings, household and per-member

14 Claude Project Setup for Build

The build lives in a dedicated Claude project — not this document and not a general conversation. The project is structured so that each build session starts with Bruce knowing the full architecture without re-explaining context.

Project Instructions

Paste this entire document into the Claude project instructions. Every chat inside the project starts with Bruce loaded with the full spec — stack, decisions, data model, build order, everything.

Chat Structure Inside the Project

Phase chats	One chat per build phase. Phase 1 Auth and Database. Phase 2 Core Chat. Etc. Leave and resume freely — project context is always loaded.
Decisions log	One persistent chat where new decisions, spec changes, and things that come up during the build are recorded. Living amendment to this document.
Bruce Dev	Troubleshooting chat — paste errors, logs, configs. Separate from build chats.
Design reference	Screenshots and mockups of the finalized UI designs for implementation reference.

Rule for Resuming After a Break

Open the project, open the relevant phase chat, read the last message. If context is unclear, ask Bruce what was last completed and what comes next. He has the full spec and the conversation history — he will orient you immediately.

Bruce

Calm. Reliable. Consistent. Intelligent. Caring.

bruce.app